

实现自评测试报告

密钥交换—ADKEX—ARM—v1.0 (ICCS 2026-06 electronic submission)

依据 ICCS 《新一代商用密码算法 ARM 架构实现自评指引》（2026 年 6 月，中文版与英文版）

本报告记录 ADKEX（密钥交换（KEX，2 趟））在 ARM 架构上的基本信息、测试环境、功能正确性、性能、资源消耗与传输/存储尺寸，并附原始证据索引，仅作为可复核技术证据，不含安全性结论。测试对象：will-submit 电子提交包。测试日期：2026-06-24。

1 基本信息

- 算法名称：ADKEX (PKCKEX-142803)
- 算法类别：密钥交换（KEX，2 趟）
- 实现版本：v1.0 (ICCS 2026-06 electronic submission)
- 测试架构：ARM
- 实现形态：参考实现（可移植 ISO C99）与优化实现（arm64 NEON）
- 实例与安全级：ADKEX-128 / -256 / -512，对应 128 / 256 / 512 比特经典安全强度

表 1 参数集

实例	N	Q	K	安全级
ADKEX-128	256	3329	2	128 比特
ADKEX-256	256	3329	4	256 比特
ADKEX-512	512	7681	4	512 比特

2 测试环境

- 主测硬件：aarch64（ARMv8.2-A 级），单核测量
- 操作系统：Ubuntu 24.04 (aarch64)
- 编译器 / 构建：GNU gcc 13; bash build.sh
- 补充交叉验证：Apple Silicon（macOS 15，Apple clang）功能 KAT 复核
- 附加嵌入式：Cortex-M4（QEMU netduinoplus2 / mps2-an386，arm-none-eabi-gcc 14.2.1）—— 32 位 ARMv7E-M，不在本指引（v8.2-A）范围，仅作附加证据
- 内存占用：进程峰值常驻内存约 1 776 KB（详见第 6 节）；静态镜像大小见第 6 节

- 测量窗口 (UTC) : 2026-06-23T21:01:00Z → 2026-06-23T21:01:57Z

3 实现版本与编译配置 (备案)

3.1 参考实现 (ISO C99, 无 SIMD)

```
gcc -std=c99 -Wpedantic -Wall -Wextra -O2  
-fcommon -DDKE_FORCE_SCALAR -DDKE_MODE=<128|256|512> -DDKE_HASH=0 -DDKE_RANDOM=0
```

3.2 优化实现 (arm64 NEON)

```
gcc/clang -O3 -march=armv8.2-a -flto -fomit-frame-pointer  
-std=c99 -Wpedantic -Wall -Wextra -fcommon -DDKE_USE_AARCH64 -DDKE_USE_AARCH64_NATIVE  
-DDKE_HASH=0 -DDKE_RANDOM=0
```

附 加 Cortex-M4 : arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -mfloat-abi=hard -mfpv4-sp-d16 -O2 -ffunction-sections -fdata-sections -std=gnu99 -nostdlib -Wl,--gc-sections。

- 目标指令集: ARMv8.2-A + NEON (Advanced SIMD; 未使用 SVE 与 SHA3/crypto 扩展)
- 随机数方法: 官方 API_PKC 确定性 DRNG (DKE_RANDOM=0), KAT 种子取自 DRNG, 可复现。
- 杂凑 / XOF: 仅 SM3, 经官方 auxfunc (sm3hash/pseudoXOF/pseudohash), DKE_HASH=0; 不构建替代杂凑。
- 第三方依赖: ARM64 NEON 的 NTT / 多项式优化汇编改编自 mlkem-native 项目 (Arm Limited、Hanno Becker、Amin Abdulrahman、Matthias Kannwischer 等; Apache-2.0 OR ISC OR MIT), 属外部第三方、不在 B.3 原创权属范围; Cortex-M4 的 Plantard NTT 汇编为提案组成员 Junhao Huang (黄俊豪) 所作 (ePrint 2022/956, Apache-2.0), 属 B.3 原创。可移植参考实现为纯 ISO C99, 无任何第三方代码。许可证见 Implementations/LICENSES/。
- 受保护文件: drng.c/.h、auxfunc.c/.h、KAT_*.c 逐字节未改。

4 功能测试 (KAT)

arm64: 参考 + AArch64_NEON, 3 实例。Ubuntu gcc 6 / 6 通过; macOS clang 6 / 6 通过 (KAT 均与 Test_Vectors 逐字节一致, 参考 ≡ NEON)。附加 Cortex-M4 自测 3 / 3 通过。

5 性能测试 (平均时间 + 平均周期)

平均值（DKE_RANDOM=0，种子取自确定性 DRNG）。KEM 取 N=300，KEX 取 N=128（均 ≥ 100 ，符合指引 §3.3）。每次迭代以 clock_gettime(CLOCK_MONOTONIC) 计挂钟时间，全程单核。

注：本 arm64 平台无稳定的用户态周期计数器（PMU 受限），故平均周期记为 NA，以平均挂钟微秒（ μs ）为准。

表 2 完整握手平均时间（ μs ）

实例	参考	NEON
ADKEX-128	359.79	281.64
ADKEX-256	990.97	806.69
ADKEX-512	3810.66	3025.32

表 3 Cortex-M4（周期，附加证据）

实例	init_b	pass1	pass2	derive
ADKEX-128	53 793	116 727	156 123	95 441
ADKEX-256	167 126	349 893	424 704	231 471
ADKEX-512	588 118	1 239 101	1 453 401	810 987

优化加速集中在 NTT / 多项式运算（手写汇编）。杂凑为经 auxfunc 的 SM3 标量实现（合规要求），故端到端加速适中。

6 资源消耗（静态镜像 + 峰值内存）

静态镜像大小由 GNU size 报告（text+data+bss 之和，单位字节），反映代码+常量+静态数据的存储占用；峰值常驻内存由 getrusage(RUSAGE_SELF).ru_maxrss 报告（单位 KB），反映运行期内存占用。单核、单进程。

表 静态镜像与峰值内存

实例	参考 静态	NEON 静态	峰值内存
ADKEX-128	NaN B	NaN B	1 776 KB
ADKEX-256	NaN B	NaN B	1 776 KB
ADKEX-512	NaN B	NaN B	1 776 KB

说明：优化实现因内联手写汇编与 -flt0 展开，静态镜像显著大于参考实现；峰值常驻内存存在各实例间基本持平（约 1.8 MB），主要为进程基线与栈上密钥材料。

7 传输与存储尺寸（字节）

共享密钥 32 B（128/256）/ 64 B（512）；消息 M1 = 1 600 / 3 168 / 6 528 B，M2 = 800 / 1 600 / 3 136 B（128/256/512）；交互 2 轮。

8 结论

ADKEX 全部三个实例在 ARM 架构上功能正确（KAT 与 Test_Vectors 逐字节一致，参考 $\equiv$ 优化），可由提交的自动化构建脚本可复现地构建。平均时间、静态镜像与峰值内存、目标指令集、编译参数、随机数方法、杂凑后端均已如上备案。

9 原始证据索引

下列原始数据与脚本随自评估材料包提交，可据以独立复现本报告全部结论：

- **3-Assessment-Code-and-Data/Raw-Evidence/assess_247.txt** —— arm64 Ubuntu gcc 性能/内存原始测量输出（含起止时间戳）
- **3-Assessment-Code-and-Data/Raw-Evidence/assess_143.txt** —— arm64 macOS clang 功能 KAT 交叉验证输出
- **3-Assessment-Code-and-Data/run_assess.sh** —— 性能/内存/KAT 一体化测量脚本
- **2-Algorithm-Source-Code/Implementations/.../build.sh** —— 各实例确定性构建脚本（功能 KAT 由此产生）
- **3-Assessment-Code-and-Data/Test_Vectors/** —— 官方比对基准向量（KAT 逐字节比对对象）

所有时间戳为 UTC；测量主机、编译器与窗口见第 2 节。